

Quản lý bộ nhớ

Tuesday, April 7, 2020 4:29 PM

Phương pháp đánh địa chỉ bộ nhớ

Địa chỉ tuyến tính

Các byte trong bộ nhớ n bytes được đánh địa chỉ tuyến tính từ 0 .. $n-1$.

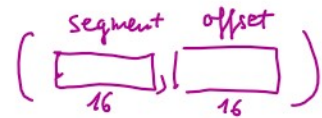
Địa chỉ phân đoạn

Địa chỉ được phân làm 2 phần Địa chỉ đoạn (**segment**) và Địa chỉ thành phần (**offset**).

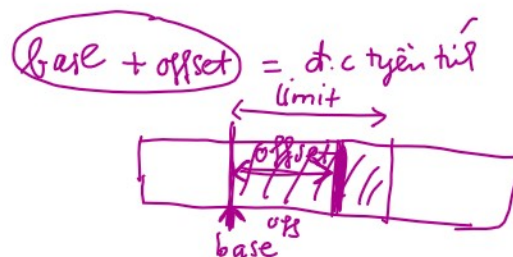
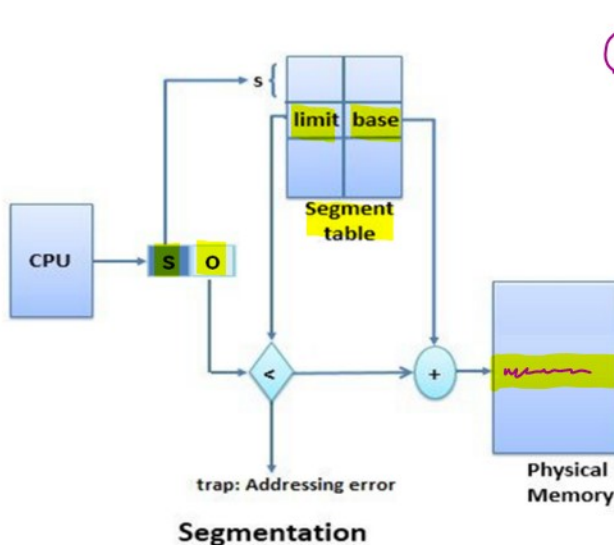
- Địa chỉ segment dùng làm chỉ số trỏ tới 1 bản ghi trong Segment Table
- Dùng trường Base làm địa chỉ mốc cơ sở. Kích thước của đoạn bộ nhớ bị giới hạn bởi trường limit.
- Nếu địa chỉ $offset \leq limit$ thì đây là một địa chỉ hợp lệ và địa chỉ tuyến tính được xác định bởi $base\ address + offset\ address$
- Nếu $offset > limit$ thì báo lỗi truy cập bộ nhớ không hợp lệ

8 bit 
16  $2^{16} B = 64 KB$

$\lceil \log_2 n \rceil$ bit địa chỉ

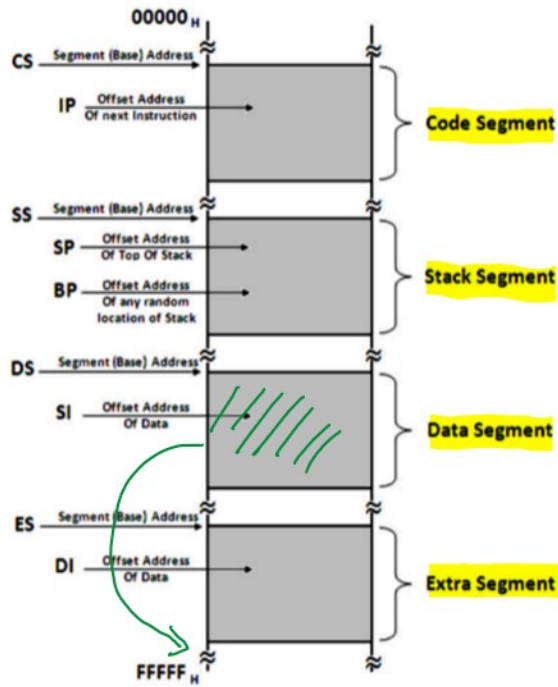


→ đ. c tuyến tính



Chương trình được phân thành các đoạn khác nhau (thực hiện bởi trình biên dịch): mã chương trình, dữ liệu, ngăn xếp, dữ liệu cấp phát thêm. Nếu truy cập tới bộ nhớ trong cùng 1 đoạn thì chỉ cần sử dụng địa chỉ offset. Có thể dễ dàng dịch chuyển một đoạn bộ nhớ tới vùng nhớ khác mà chỉ cần thay đổi địa chỉ cơ sở của đoạn.

Phân đoạn được sử dụng trong kiến trúc Intel x86 cho tới các BVXL 32 bit. Từ BVXL Intel x86 64 bit, phương pháp đánh địa chỉ phân đoạn không được sử dụng nữa mà chỉ hỗ trợ đảm bảo tính tương thích ngược.



quora.com

segment: offset

CS: IP
16 / 32

Seg : offs
↓
Seg : offs

SS: SP
BP

DS: SI

ES: DI

Ví dụ về phân đoạn trong BVXL 8086

Địa chỉ sử dụng là 16 bit. Mỗi đoạn tối đa là 64KB.

Địa chỉ tuyến tính = segment × 16 + offset

Kích thước bộ nhớ tối đa là 16 segments, tức là 1MB.

Intel x86

$$2^{16} \times 2^4 = 2^{20} \text{ Segment } 2^{16} \text{ B} = 64 \text{ KB}$$

$$2^{20} \text{ B} = 1 \text{ MB}$$

$$\text{seg} + \text{offset} = \text{d.chi}$$

$$16 \quad 16 \quad 16 \text{ bit}$$

$$0 \times 16 = 0$$

$$1 \times 16 = 16$$

$$2 \times 16 = 32$$

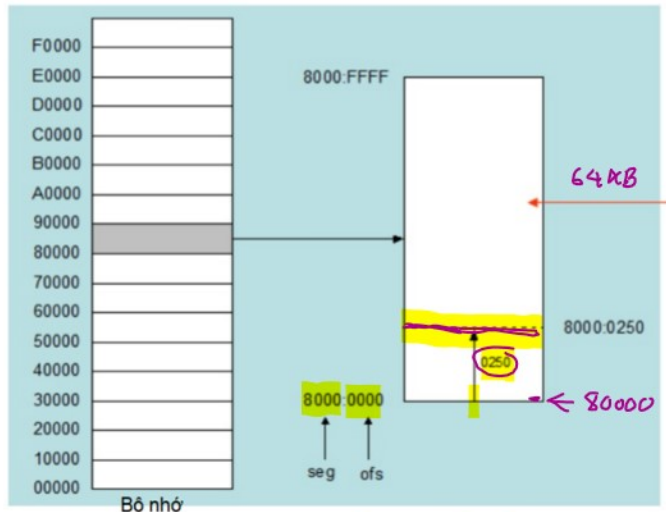
$$16 \times 64 \text{ KB} = 1 \text{ MB}$$

seg: offs 8000 : 0000

8000_H: 0250_H

$$\Rightarrow 8000_{\text{H}} \times 10_{\text{H}} + 0250$$

$$\begin{array}{r} 8000 \\ 0250 \\ \hline 80250_{\text{H}} \end{array} \quad 20 \text{ bit}$$

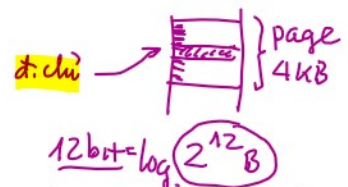


Segmentation

Phân trang Paging

Bộ nhớ được quản lý cấp phát theo đơn vị trang nhớ, kích thước xác định bởi phần cứng, thường là 4KB.

Một địa chỉ tuyến tính có thể được tách thành 2 phần: địa chỉ trang và địa chỉ thành phần trong trang.



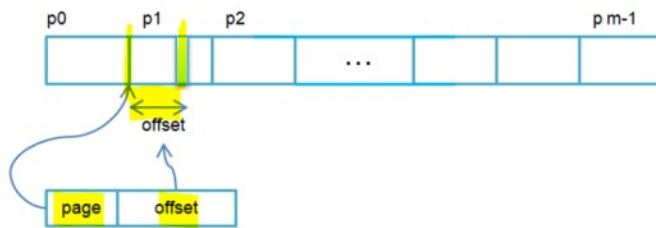
$$12 \text{ bit} = \log_2(2^{12} \text{ B})$$

Một địa chỉ tuyến tính có thể được tách thành 2 phần: địa chỉ trang và địa chỉ thành phần trong trang.

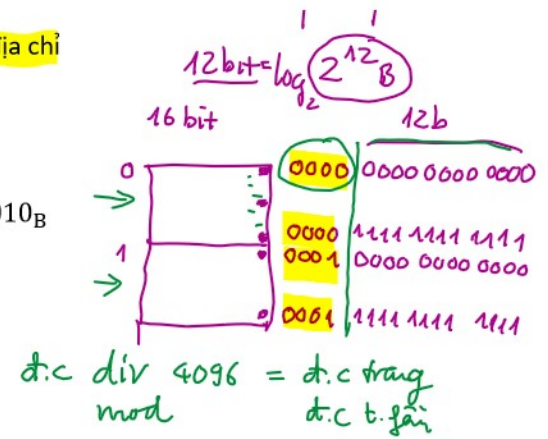
Ví dụ: địa chỉ 4098 trở tới trang số 1, byte thứ 3.

$4098_D = 0001\ 0000\ 0000\ 0010_B$

Địa chỉ trang là 0001_B , địa chỉ thành phần trong trang là $0000\ 0000\ 0010_B$



Việc ứng dụng phân trang trong cấp phát/giải phóng bộ nhớ và quản lý bộ nhớ ảo sẽ được thảo luận trong phần dưới đây của chương.



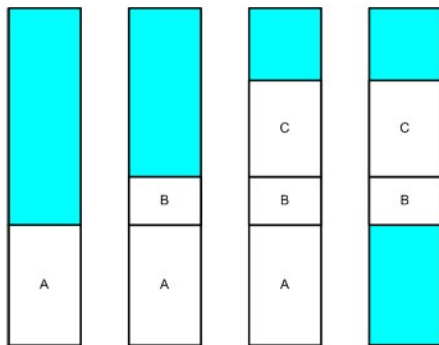
Sequestration
paging

Quản lý cấp phát và giải phóng bộ nhớ

Phân bổ bộ nhớ cho các tiến trình

Các tiến trình được tải vào bộ nhớ, thực hiện và kết thúc, bộ nhớ được giải phóng, dẫn tới tình trạng phân mảnh. Vấn đề phân mảnh còn nghiêm trọng hơn khi một số trang nhớ ít sử dụng của một tiến trình phải bị đẩy ra bộ nhớ ngoài lấy chỗ cho các tiến trình khác.

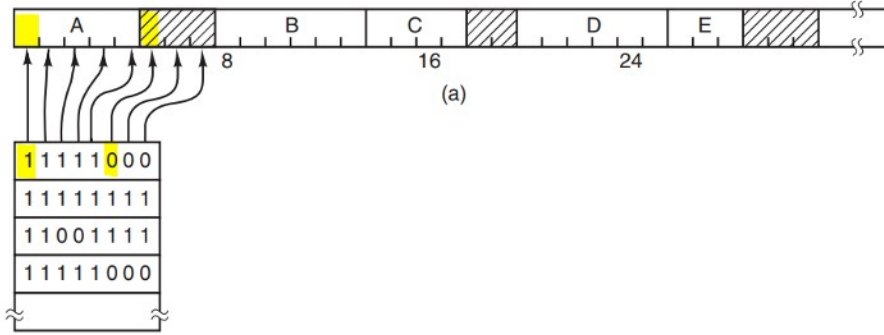
Vấn đề đặt ra là cần có các thuật toán cấp phát và giải phóng bộ nhớ hiệu quả cho việc thực thi các tiến trình.



Quản lý bộ nhớ bằng bản đồ bit

Bộ nhớ được phân thành các trang (4KB), là đơn vị cấp phát cho các tiến trình. Một phần bộ nhớ được sử dụng làm bản đồ bit để lưu trạng thái của các trang nhớ

- bit $i=1$ biểu diễn cho trạng thái trang nhớ i đã được cấp phát
- bit $i=0$ biểu diễn cho trạng thái trang nhớ i còn rỗi



Thao tác cấp phát:

- Yêu cầu của tiến trình được làm tròn lên thành k trang nhớ
- Tìm chuỗi k bit 0 trong bản đồ bit
- Đảo k bit 0 thành k bit 1

số trang nhớ
↑
 $O(n \cdot k)$

Thao tác giải phóng:

- Tiến trình kết thúc, trả lại k trang nhớ
- Đặt lại k bit tương ứng với các trang nhớ của tiến trình thành bit 0

$O(k)$

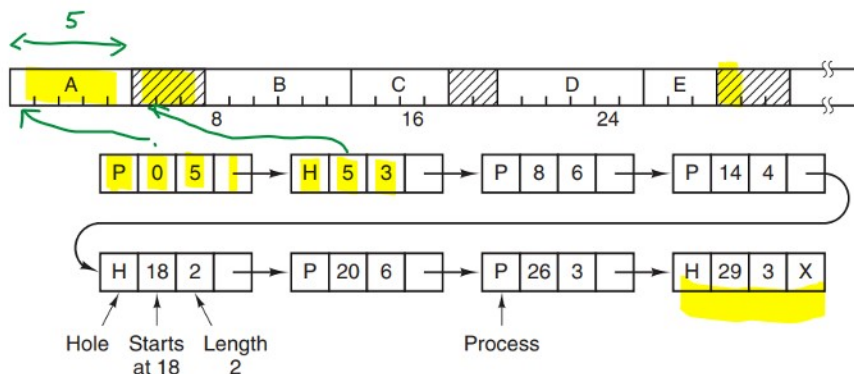
Nhận xét:

- Cần hỗ trợ của phần cứng: tìm kiếm chuỗi bit
- Mỗi trang biểu diễn bởi 1 bit, và quản lý độc lập từng trang

Quản lý bộ nhớ bằng danh sách móc nối

Bộ nhớ được phân thành các trang (4KB), là đơn vị cấp phát cho các tiến trình. Một phần bộ nhớ được sử dụng để lưu danh sách móc nối các vùng nhớ

- Mỗi vùng nhớ gồm các trang nhớ có cùng trạng thái (rỗi/bận)
- Vùng nhớ rỗi được dán nhãn **H (Hole)**, bận được dán nhãn **P (Process)**
- Mỗi phần tử của dsmn gồm 4 trường:
 - o Nhãn của vùng nhớ
 - o Địa chỉ trang nhớ bắt đầu của vùng
 - o Số trang thuộc vùng nhớ
 - o Con trỏ tới phần tử dsmn tiếp theo



4MB 1000 trang
4KB

Thao tác cấp phát:

- Yêu cầu của tiến trình được làm tròn lên thành k trang nhớ
- Tìm trong dsmn 1 phần tử có nhãn H, số trang trong vùng $\geq k$
- Tách phần tử đó thành 2 phần tử mới, một trỏ tới vùng nhớ cấp cho tiến trình, phần tử còn lại thì trỏ tới vùng rỗi còn lại

số trang nhớ

$O(n)$

Thao tác giải phóng:

Thao tác giải phóng:

- Duyệt dsmn tới khi gặp phần tử trở tới vùng nhớ được giải phóng
- Chuyển nhãn P thành H
- Hợp nhất 2 phần tử dsmn liền kề, nếu có thể, để tạo được vùng H lớn nhất

$O(n)$

Nhận xét:

- Số phần tử dsmn trong thực tế ít hơn số trang nhớ rất nhiều lần
- Không hiệu quả khi bộ nhớ bị phân mảnh nhiều

Quản lý bộ nhớ theo khối có kích thước bội mũ 2

Bộ nhớ ban đầu gồm 1 khối kích thước **bội mũ 2 (buddy)**. Yêu cầu cấp phát của các tiến trình phải được làm tròn lên thành 1 khối buddy.

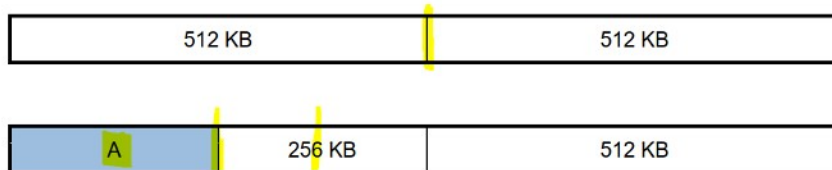
Thao tác cấp phát: chia đôi buddy hiện tại cho tới khi tìm được buddy đủ nhỏ cho tiến trình

Thao tác giải phóng: hợp nhất với các buddy liền kề để tái tạo lại buddy lớn nhất có thể

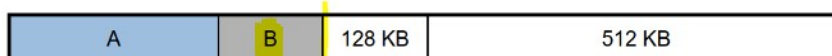
Ví dụ: Bộ nhớ 1024KB



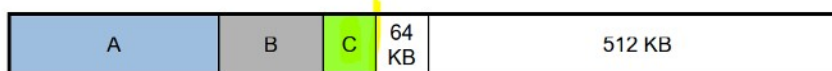
Tiến trình A yêu cầu **200KB**, được làm tròn lên thành **256KB**



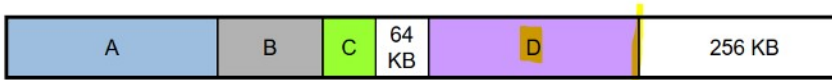
Tiến trình B yêu cầu **120KB**, được làm tròn lên thành **128KB**



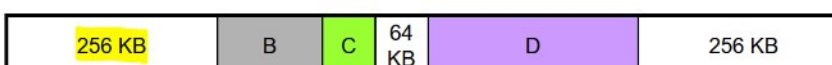
Tiến trình C yêu cầu **40KB**, được làm tròn lên thành **64KB**



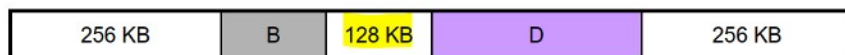
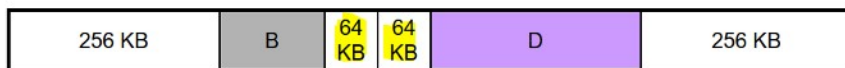
Tiến trình D yêu cầu **130KB**, được làm tròn lên thành **256KB**



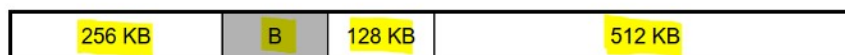
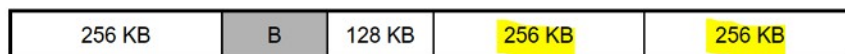
Tiến trình A kết thúc



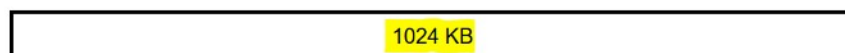
Tiến trình C kết thúc



Tiến trình D kết thúc



Tiến trình B kết thúc



Nhận xét:

- Thao tác cấp phát và giải phóng đều rất nhanh, có độ phức tạp $\log_2 n$
- Nhược điểm: lãng phí bộ nhớ vì phải làm tròn kích thước lên bội mũ 2. Phương pháp này phù hợp với bộ nhớ ngoài

Các chiến lược cấp phát

First Fit: Tìm từ đầu bộ nhớ tới khi gặp khối bộ nhớ đầu tiên đủ cấp phát cho tiến trình

Next Fit: Tìm từ vị trí thành công trước đó tới khi gặp khối bộ nhớ đầu tiên đủ cấp phát cho tiến trình

Worst Fit: Tìm khối bộ nhớ có kích thước lớn nhất đủ cấp phát cho tiến trình

Best Fit: Tìm khối bộ nhớ có kích thước nhỏ nhất đủ cấp phát cho tiến trình

Ví dụ:

First fit

Requests: A – 12 KB B – 10 KB C – 9 KB



Next fit

Requests: A – 12 KB B – 10 KB C – 9 KB



Worst fit

Requests: A – 12 KB B – 10 KB C – 9 KB



Best fit

Requests: A – 12 KB B – 10 KB C – 9 KB



Nhận xét:

- Best fit: tạo các phân mảnh quá nhỏ để tận dụng cho các tiến trình khác
- Worst fit: luôn phân mảnh các khối nhớ lớn, do vậy bộ nhớ có thể không còn khối lớn để cấp cho các tiến trình có nhu cầu cao
- Cả best và worst fit đều đòi hỏi duyệt toàn bộ để tìm được khối nhớ phù hợp
- First fit và next fit chỉ tìm khối nhớ đầu tiên thỏa mãn => nhanh hơn. Next fit cho kết quả tốt hơn trong giai đoạn đầu.

Bộ nhớ ảo

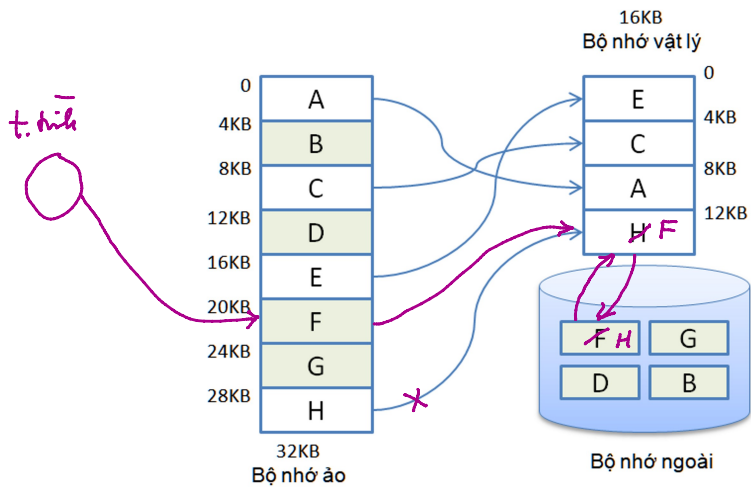
Khái niệm bộ nhớ ảo

Bộ nhớ ảo là một không gian địa chỉ logic, được chia thành các trang nhớ mà một số trang được ánh xạ trực tiếp tới các trang trong bộ nhớ vật lý, và các trang còn lại có nội dung được lưu trong bộ nhớ ngoài.

- Nội dung trong các trang đang được ánh xạ vào bộ nhớ vật lý (RAM) có thể truy cập được trực tiếp sau khi đã chuyển đổi từ địa chỉ ảo sang địa chỉ thực
- Nội dung trong các trang lưu ở bộ nhớ ngoài, không thể truy cập trực tiếp mà cần phải được tải vào trong bộ nhớ vật lý và thiết lập ánh xạ địa chỉ giữa trang nhớ ảo và thực. Việc tải 1 trang vào bộ nhớ thực có thể đòi hỏi việc hoán đổi nội dung với bộ nhớ ngoài.

Vì sao cần bộ nhớ ảo: Bộ nhớ ảo có kích thước lớn hơn bộ nhớ vật lý, nhằm mục đích

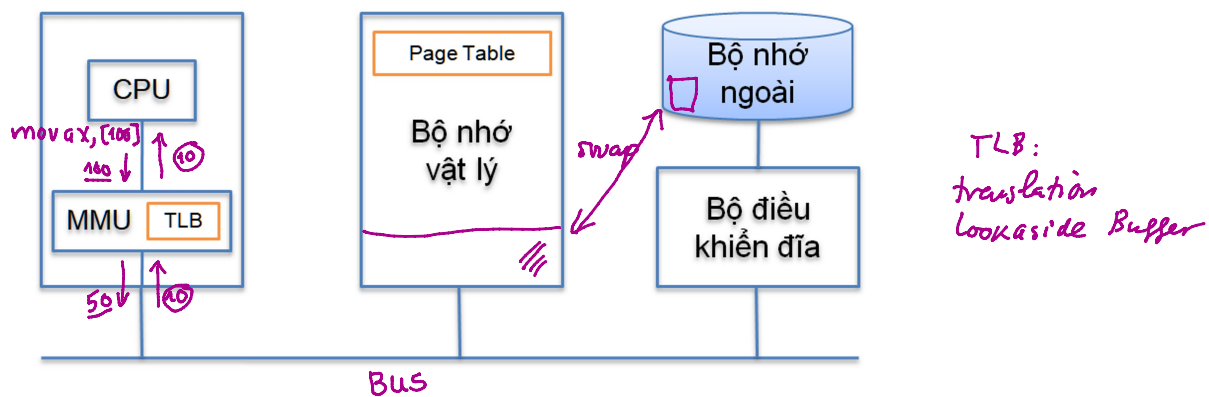
- Tạo không gian nhớ lớn hơn kích thước bộ nhớ thực cho các tiến trình
- Đánh đổi hiệu suất hoán đổi dữ liệu với kích thước không gian nhớ



Memory Management Unit - MMU

Cài đặt bộ nhớ ảo đòi hỏi sự phối hợp giữa cả phần cứng và phần mềm (hệ điều hành). MMU là thiết bị được đặt giữa CPU và bus nối với các thiết bị khác, thực hiện nhiệm vụ:

- Chuyển đổi địa chỉ ảo sang địa chỉ thực
- Phối hợp với hệ điều hành để thực hiện hoán đổi trang giữa bộ nhớ ngoài với bộ nhớ vật lý, thiết lập ánh xạ địa chỉ



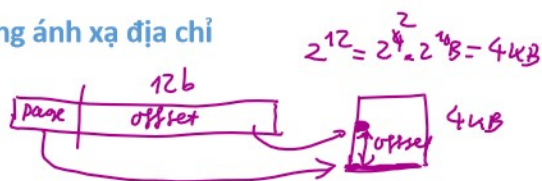
Các bước phối hợp như sau:

- CPU gửi yêu cầu truy cập tới 1 địa chỉ ảo đang được ánh xạ tới một trang bộ nhớ thực. MMU thực hiện chuyển đổi sang địa chỉ thực
 - o Tra bảng lưu đệm (cache) Translation Lookaside Buffer (TLB) để chuyển đổi
 - o Nếu không tìm thấy trong TLB, MMU tra bảng ánh xạ địa chỉ Page Table để chuyển đổi địa chỉ, sau đó cập nhật lại quan hệ mới vào TLB
- Trong trường hợp địa chỉ CPU yêu cầu đang không ứng với nội dung trang nhớ thực nào, khi đó MMU không thể chuyển đổi địa chỉ
 - o MMU tạo ra sự kiện Page Fault và chuyển tiếp sang cho module Page Supervisor của hệ điều hành xử lý tiếp
 - o Page Supervisor thực hiện hoán đổi trang giữa bộ nhớ ngoài với bộ nhớ vật lý, cập nhật bảng ánh xạ địa chỉ Page Table
 - o Page Supervisor chuyển tiếp lại điều khiển cho MMU thực hiện lại việc chuyển đổi địa chỉ.

Các khái niệm Page Table, TLB và các thuật toán hoán đổi trang nhớ sẽ được thảo luận trong phần dưới đây.

Chuyển đổi địa chỉ ảo sang địa chỉ thực sử dụng bảng ánh xạ địa chỉ

Một địa chỉ bộ nhớ có thể được phân làm 2 phần:
địa chỉ trang + địa chỉ thành phần trong trang.



Đối với trang nhớ 4KB, địa chỉ thành phần dài 12 bits, phần trước đó là địa chỉ trang.

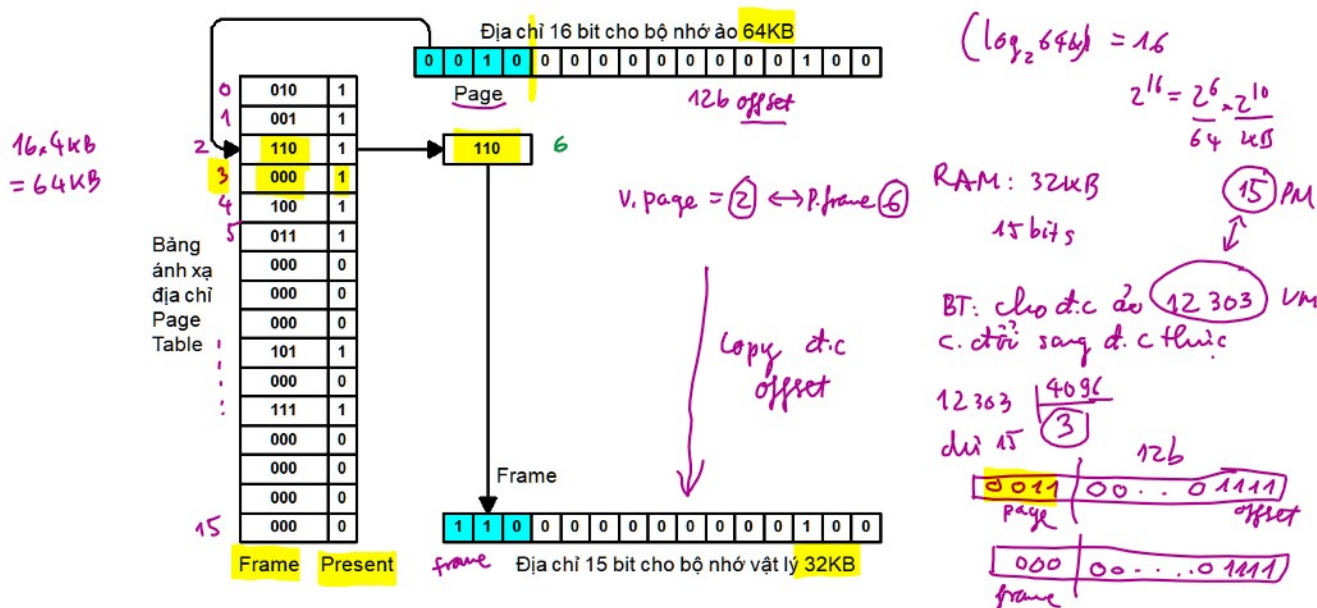
Do 1 trang nhớ ảo được ánh xạ vào 1 trang nhớ thực nên việc chuyển đổi địa chỉ thực ra là chuyển đổi phần địa chỉ trang nhớ ảo sang địa chỉ trang nhớ thực và giữ nguyên phần địa chỉ thành phần.

Quy ước: trang nhớ ảo được gọi là **page**, trang nhớ thực được gọi là **frame** (page frame).

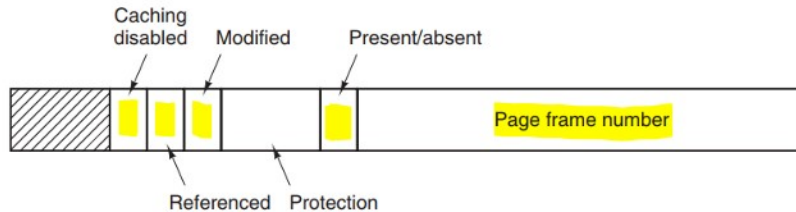
Bảng ánh xạ địa chỉ (Page Table) gồm 2 cột:

- Frame: chứa địa chỉ trang nhớ thực tương ứng
- Present: quan hệ địa chỉ còn hợp lệ hay không; ==1 là còn; ==0 là không

Ví dụ: bộ nhớ ảo 64KB, bộ nhớ thực 32KB, quan hệ giữa trang nhớ ảo và thực được cho trong bảng ánh xạ địa chỉ.



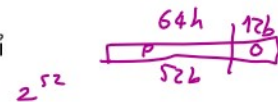
Một bản ghi đầy đủ trong bảng Page Table



1

- Page Frame Number: chứa địa chỉ trang nhớ thực
- Present/Absent: trang có đang nằm trong bộ nhớ vật lý không, nếu không thì lỗi page fault xuất hiện.
- Protection: quyền truy cập trang (Read, Write, Execute)
- Bit M: Modified. Nội dung trang đã bị thay đổi chưa
- Bit R: Referenced. Trang đã được dùng tới chưa?
- Caching disabled: Không cho phép CPU lưu đệm trang nhớ

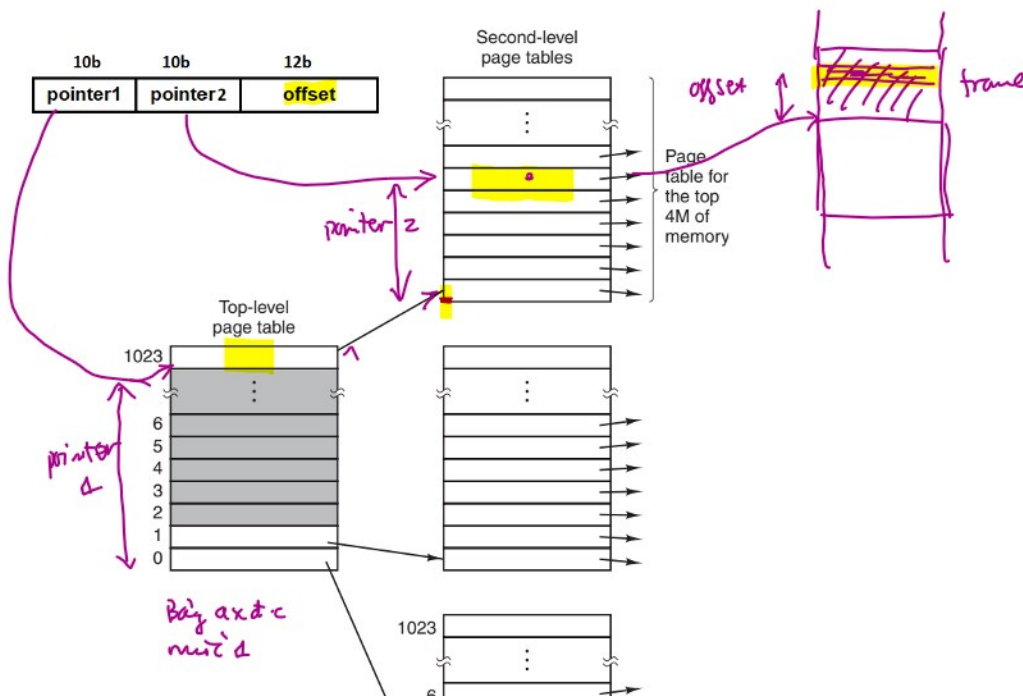
Nhận xét: Nếu chúng ta chỉ có 1 con trỏ địa chỉ trang cho tới bảng ánh xạ địa chỉ thì kích thước của bảng ánh xạ địa chỉ có thể lớn

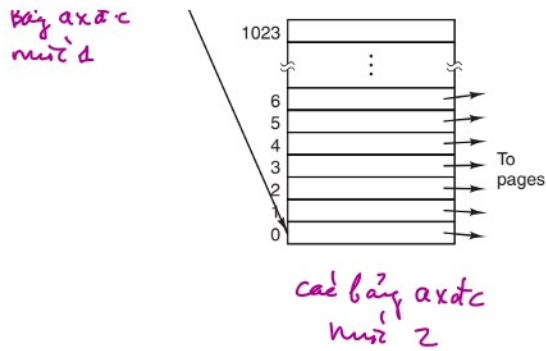


Bảng ánh xạ địa chỉ nhiều mức

Địa chỉ được chia làm nhiều mức con trỏ trang. Hình dưới đây ví dụ với địa chỉ 32 bit, được tách làm 2 mức trỏ trang

- Con trỏ trang 1, trỏ tới dòng trong bảng ánh xạ địa chỉ mức 1
- Mỗi dòng trong bảng ánh xạ địa chỉ mức 1 trỏ tới 1 bảng ánh xạ địa chỉ mức 2
- Con trỏ trang 2, trỏ tới 1 dòng trong bảng ánh xạ địa chỉ mức 2 vv..
- Tại mức cuối cùng, mỗi dòng trong bảng ánh xạ địa chỉ trỏ tới một trang nhớ thực





Nhận xét:

- Chia nhỏ được bảng ánh xạ địa chỉ thành nhiều bảng con
- Yêu cầu nhiều bước tra cứu chuyển đổi địa chỉ hơn so với bảng 1 mức

Translation Lookaside Buffer / Bộ nhớ liên kết (Associative Memory)

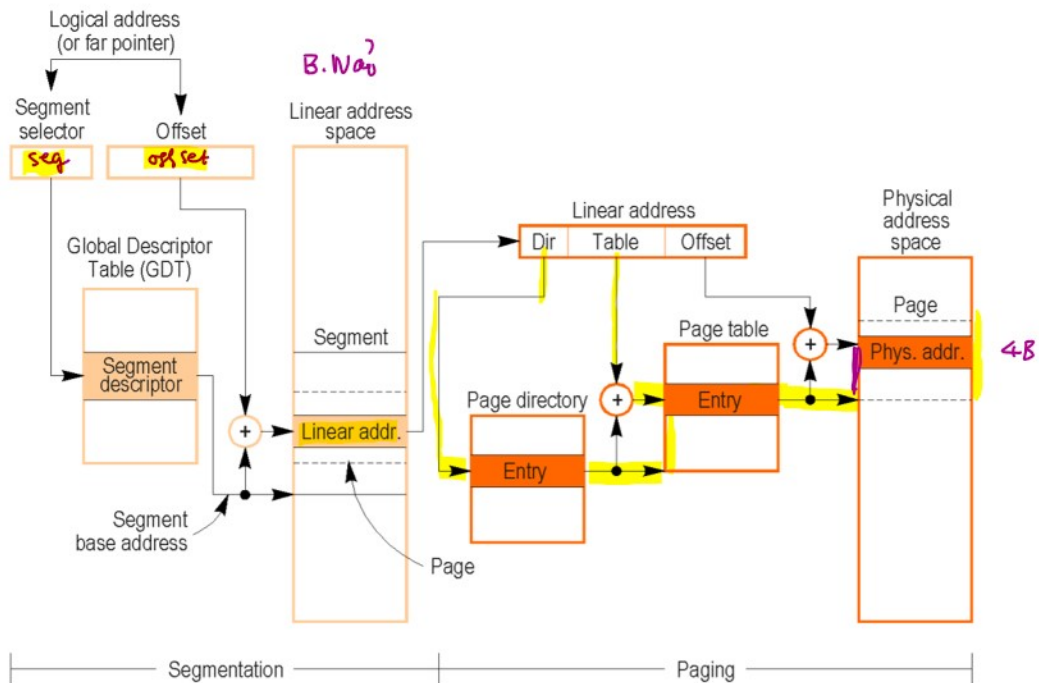
TLB là bộ nhớ đệm của MMU gồm các thanh ghi độc lập cho phép tìm kiếm song song, lưu các quan hệ (*trang nhớ ảo, trang nhớ thực*) thường xuyên được sử dụng. Cấu trúc bảng TLB

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

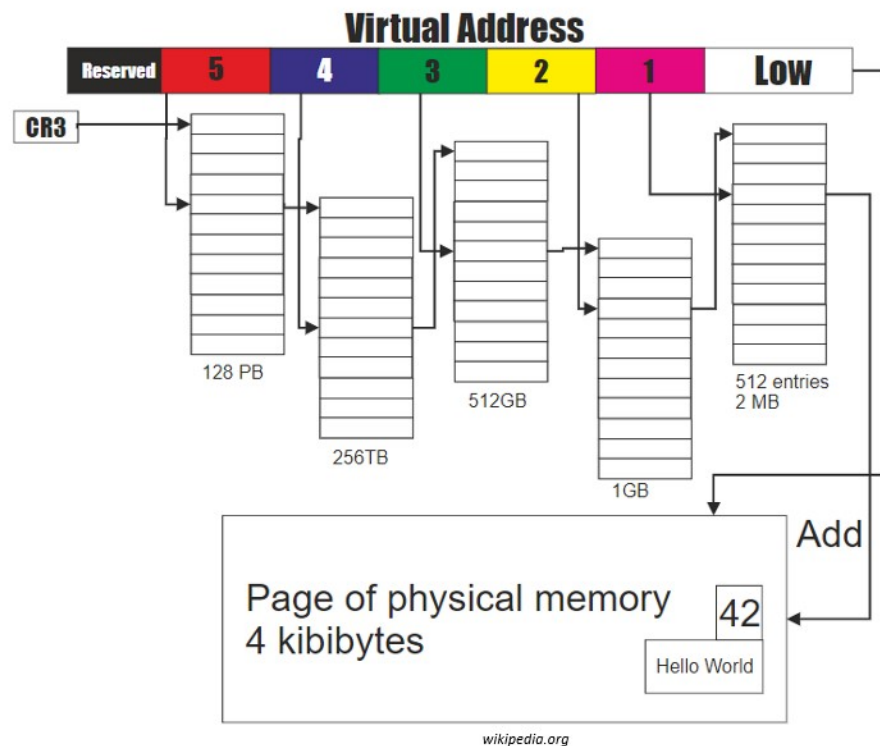
Khi có 1 yêu cầu chuyển đổi địa chỉ, MMU sẽ

- Tìm kiếm trên TLB
- Nếu không tìm thấy thì sử dụng Page Table để phân giải địa chỉ. Sau khi phân giải được thì cập nhật quan hệ mới vào TLB
- Nếu không phân giải được bởi Page Table, tạo sự cố Page Fault để hệ điều hành thực hiện hoán đổi trang giữa bộ nhớ vật lý với bộ nhớ ngoài, cập nhật lại Page Table và chuyển điều khiển để MMU thực hiện lại việc chuyển đổi địa chỉ.

Tóm tắt lại về phân đoạn và phân trang trong Intel x86



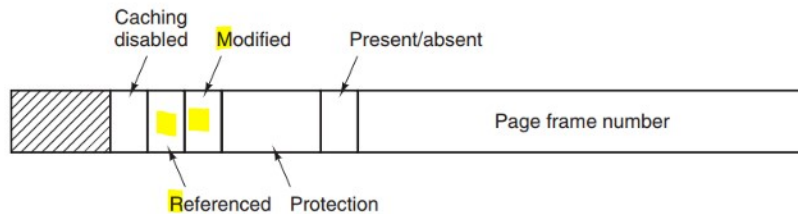
Lưu ý: Kể từ BVXL Intel x86 64 bit, địa chỉ phân đoạn không còn được sử dụng, mà chỉ hỗ trợ để tương thích ngược. Intel x86 64 bit sử dụng cơ chế phân trang 5 mức.



Các thuật toán hoán đổi trang nhớ

Thuật toán NRU

NRU - Not Recently Used:



Mỗi trang nhớ được gắn kèm 2 bit thuộc tính

- Bit **R** (Referenced): bật khi trang nhớ được dùng tới
- Bit **M** (Modified): bật khi trang nhớ có nội dung bị thay đổi

Các trang được phân thành 4 nhóm, dựa trên 2 bit **R, M**:

1. Not Referenced, Not Modified
2. Not Referenced, Modified
3. Referenced, Not Modified
4. Referenced, Modified

Việc hoán đổi được lựa chọn theo thứ tự nhóm 1, 2, 3, 4. Trong mỗi nhóm, chọn ngẫu nhiên 1 trang để loại.

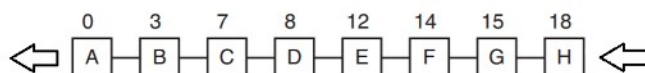
Lưu ý rằng, một trang có nội dung thay đổi nhưng không được dùng tới gần đây bị ưu tiên loại cao hơn trang chưa bị thay đổi nhưng được dùng gần đây.

Nhận xét:

- Chỉ dựa trên 2 bit thuộc tính
- Trong mỗi nhóm, loại ngẫu nhiên

Thuật toán FIFO

Các trang nhớ được sắp theo thời điểm tải vào. Trang nào cũ nhất sẽ được chọn để hoán đổi.



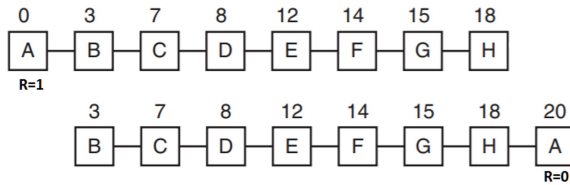
Nhận xét:

- Những trang được tải vào lâu nhất có thể là những trang được dùng thường xuyên

Thuật toán Second-Chance

Mỗi trang gắn thêm 1 bit thuộc tính R (Referenced, đặt bằng 1 khi được dùng tới).

Nếu trang bị loại có bit $R=1$ thì được trao cơ hội thứ 2: xóa bit R và xếp vào cuối hàng đợi như một trang mới được tải vào.

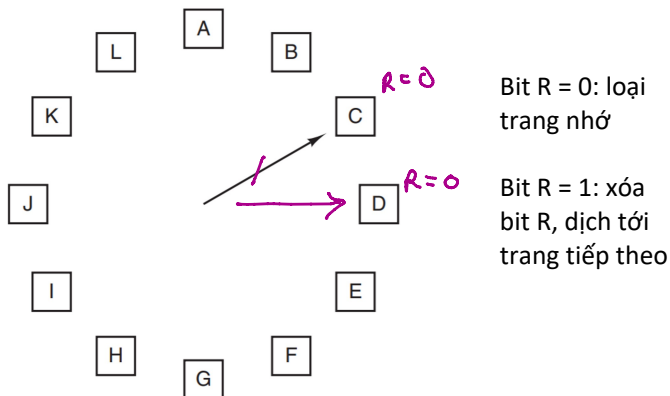


Nhận xét:

- Đã sử dụng bit R để tránh loại các trang được dùng tới. Tuy nhiên các trang có tần suất sử dụng thường xuyên nhưng không liên tục vẫn có thể bị loại
- Cần duy trì và quản lý các trang nhớ theo danh sách móc nối

Thuật toán CLOCK

Các trang nhớ được xếp thành vòng theo thứ tự địa chỉ trang. Ban đầu kim chỉ số trở tới trang tải vào đầu tiên.



Nhận xét:

- không phải duy trì ds mn
- Tuy nhiên vẫn có thể loại các trang dùng thường xuyên nhưng không liên tục

Thuật toán LRU dùng phần cứng

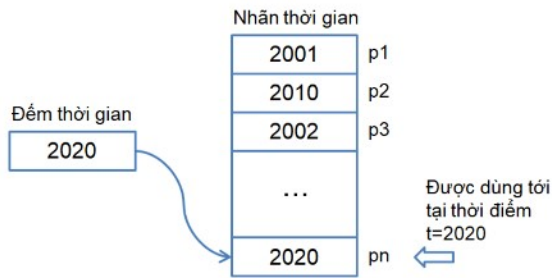
LRU - Least Recently Used là tiêu chí để chọn trang hoán đổi. Hệ điều hành hy vọng rằng những trang đã lâu không được dùng tới sẽ là những trang có xác suất thấp được dùng ngay trong các bước tiếp theo. Các cài đặt cần sự hỗ trợ của phần cứng.

Cài đặt 1:

Duy trì 1 thanh ghi 64b đếm thời gian hiện tại t , mỗi trang nhớ gắn kèm 1 nhãn thời gian gần nhất được sử dụng tới.

Khi trang nhớ i được sử dụng tới tại thời điểm t , nhãn thời gian của trang nhớ sẽ được cập nhật là t .

Khi xảy ra sự kiện Page Fault, trang có giá trị nhãn thời gian nhỏ nhất, tức là trang lâu không được dùng tới nhất, sẽ bị hoán đổi.



Cài đặt 2:

Duy trì ma trận bit cỡ $n \times n$, trong đó n là số lượng trang nhớ.

Khi trang nhớ i được dùng tới, ma trận được cập nhật như sau

- Bật các bit thuộc hàng i
- Tắt các bit thuộc cột i

Tại thời điểm xảy ra sự kiện Page Fault, trang nhớ có giá trị bit theo hàng nhỏ nhất sẽ bị hoán đổi.

0	1	1	1
0	0	0	0
0	0	0	0
0	0	0	0

Trang 0 dùng tới

0	0	1	1
1	0	1	1
0	0	0	0
0	0	0	0

Trang 1 dùng tới

0	0	0	1
1	0	0	1
1	1	0	1
0	0	0	0

Trang 2 dùng tới

Nhận xét:

- Ngoài 2 bit R, M trong bảng ánh xạ địa chỉ của mỗi trang, cần thêm không gian nhớ để lưu trạng thái nhần thời gian sử dụng của mỗi trang
- Để tìm dòng/hàng có giá trị nhỏ nhất, cần sự hỗ trợ của phần cứng

Thuật toán Aging dùng phần mềm

Aging được sử dụng để mô phỏng LRU bằng phần mềm.

Mỗi trang nhớ được gắn kèm 1 thanh ghi l bits lưu trạng thái bit R của l nhịp đồng hồ gần nhất.

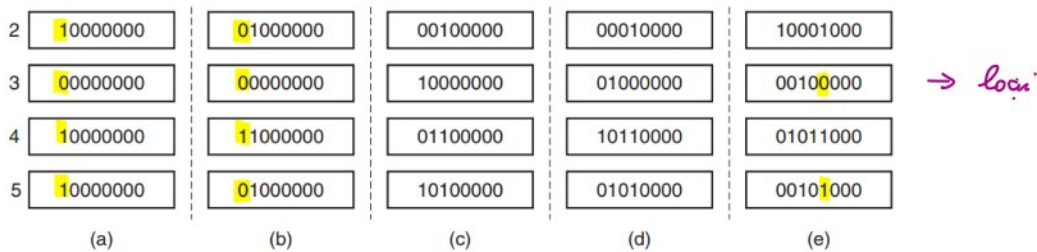
Khởi tạo, tất cả các thanh ghi được gán bằng 0.

Sau mỗi nhịp đồng hồ, các thanh ghi được dịch phải 1 bit và bit trái nhất được điền vào là bit R của trang nhớ tương ứng.

Tại thời điểm Page Fault, trang có giá trị thanh ghi nhỏ nhất là trang bị loại.

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page $l = 8$					
0	10000000	10000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00010000	10001000
3	00000000	00000000	10000000	01000000	00100000

→ loại



Nhận xét:

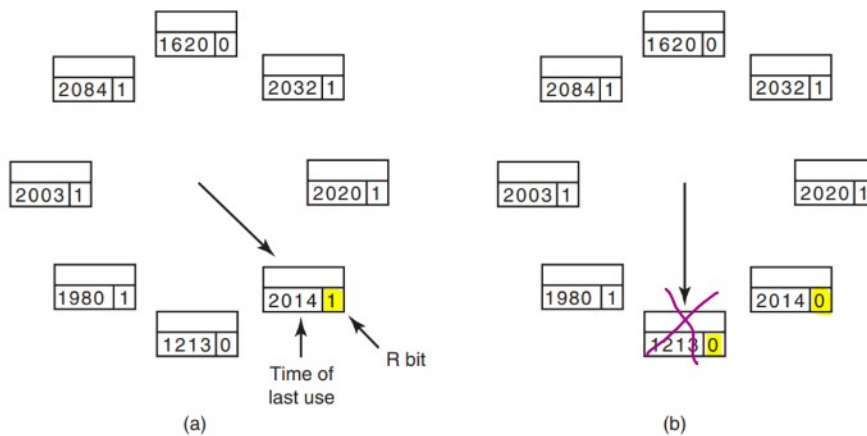
- Cho phép nhớ lại được lịch sử 1 bit trước đó
- Phải duy trì một thanh ghi cho mỗi trang nhớ
- Chi phí thực hiện khá cao: phải dịch n trang, đặt bit cao nhất; phải tìm thanh ghi có giá trị nhỏ nhất

Thuật toán WS-CLOCK

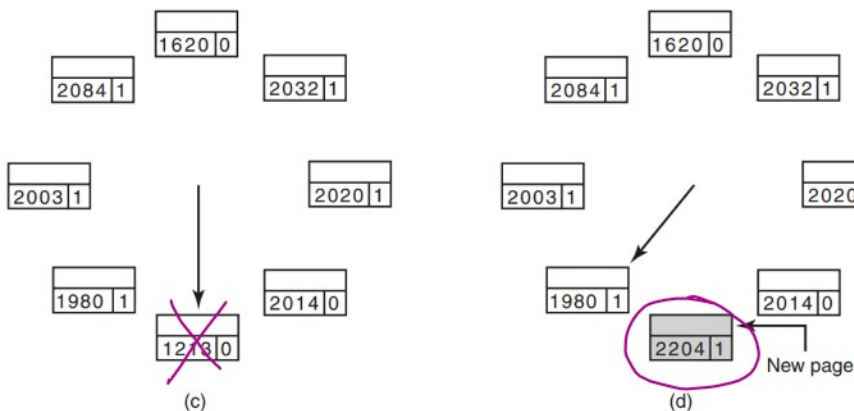
Working Set-CLOCK là thuật toán CLOCK duy trì khoảng thời gian giới hạn τ cho các tiến trình được sử dụng gần đây trong khoảng thời gian $\leq \tau$ (thuộc working set) sẽ chưa bị loại. Mỗi trang nhớ được gắn kèm 2 bit thuộc tính R và M.

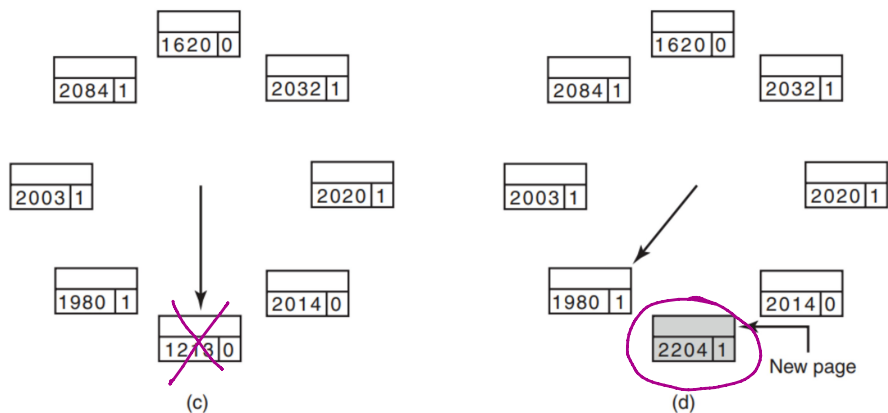
Ví dụ: giả sử thời gian hiện tại là 2204, $\tau = 100$. Những trang có nhãn thời gian từ 2104 trở lên sẽ chưa bị loại bởi thuật toán, kim chỉ số sẽ chuyển tới trang tiếp theo.

Trường hợp 1, trang bị trở tới có bit R=1, thuật toán CLOCK xóa bit R và chuyển tới xét trang tiếp theo.



Trường hợp 2, trang bị trở tới có bit R=0 và nhãn thời gian không thuộc khoảng working set τ , khi đó trang nhớ sẽ bị hoán đổi, bit R của trang mới tải được bật và kim chỉ số chuyển tới trang tiếp theo.





Vấn đề với bit M

- Nếu trang bị trở tới có bit $M=1$, cần phải ghi lại nội dung ra bộ nhớ ngoài trước khi hoán đổi. Thao tác ghi sẽ xếp vào hàng đợi chờ thực hiện, trong lúc đó kim chỉ số được chuyển tới trang tiếp theo.
- Trường hợp không tìm được ứng cử viên, kim chỉ số phải quay lại 1 vòng thì
 - Nếu đã có trang nhớ xếp trong hàng đợi chờ ghi, sớm muộn thao tác này sẽ kết thúc và trang nhớ đó sẽ bị loại
 - Nếu không thì chọn ngẫu nhiên 1 trang nhớ có bit $M=0$ và loại

Nhận xét:

- Có chi phí thực hiện thấp, khả thi bằng phần mềm
- Chỉ cần duy trì nhãn thời gian cho mỗi trang nhớ